



МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

---

Институт информационных технологий (ИИТ)  
Кафедра математического обеспечения и стандартизации информационных технологий  
(МОСИТ)

## **ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ**

по дисциплине «Тестирование и верификация программного обеспечения»

### **Практическое занятие № 2**

*ИКБО-43-23 ОГАНЕСЯН Д. К.  
КОЩЕЕВ М.И.  
ГУТКОВИЧ А.А.  
ИВАНОВ А.В.  
УЛЗЕТУЕВ А.С.*

---

(подпись)

**АССИСТЕНТ**

**ИЛЬИЧЕВ Г.П.**

---

(подпись)

**ОТЧЕТ  
ПРЕДСТАВЛЕН**

**«17»\_\_ОКТЯБРЯ\_2025 Г.**

Москва 2025 г.

## СОДЕРЖАНИЕ

|                                                |    |
|------------------------------------------------|----|
| 1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ.....       | 3  |
| 2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....                      | 4  |
| 2.1 Разработка модулей.....                    | 4  |
| 2.2 Модульное тестирование.....                | 5  |
| 2.3 Мутационное тестирование.....              | 8  |
| 3 АНАЛИЗ И ВЫВОДЫ.....                         | 12 |
| 3.1 Оценка качества тестирования.....          | 12 |
| 3.2 Анализ недостатков и их устранение.....    | 12 |
| 3.3 Итоговые выводы по результатам работы..... | 13 |

## **1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ**

Цель работы: познакомить студентов с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

## 2 ПРАКТИЧЕСКАЯ ЧАСТЬ

### 2.1 Разработка модулей

Описание функциональности: Модуль представляет собой систему конвертации единиц измерения, включающую словарь базовых единиц для длины (миллиметры, сантиметры, метры, километры) и массы (граммы, килограммы, центнеры, тонны), а также набор функций для преобразования температур между шкалами Цельсия, Фаренгейта и Кельвина. Основной функционал реализован через функцию `convert_linear` для конвертации линейных единиц и специализированные функции для температурных преобразований, при этом система обеспечивает валидацию входных данных и корректное математическое преобразование значений с использованием коэффициентов масштабирования.

*Листинг 1 — Исходный код.*

```
# Единицы измерения
UNITS = {
    "длина": {
        "миллиметр": 10 ** -3, # 0.001 метра
        "сантиметр": 10 ** -2, # 0.01 метра
        "дециметр": 10 ** -1, # 0.1 метра
        "метр": 1, # базовая единица
        "километр": 10 ** 3 # 1000 метров
    },
    "масса": {
        "грамм": 1, # базовая единица
        "килограмм": 10 ** 3, # 1000 грамм
        "центнер": 10 ** 5, # 100 000 грамм
        "тонна": 10 ** 6 # 1 000 000 грамм
    }
}

# Функция преобразования цельсия в фаренгейты
def c_to_f(c):
    return c * 9.0 + 32.0

# Функция преобразования фаренгейтов в цельсии
def f_to_c(f):
    return (f - 32.0) * 5.0 / 9.0

# Функция преобразования цельсия в кельвины
def c_to_k(c):
    return c + 273.15

# Функция преобразования кельвинов в цельсии
```

```

def k_to_c(k):
    return k - 273.15

# Функция преобразования фараенгейтов в кельвины
def f_to_k(f):
    return (f - 32.0) * 5.0 / 9.0 + 273.15

# Функция преобразования кельвинов в фараенгейты
def k_to_f(k):
    return (k - 273.15) * 9.0 / 5.0 + 32.0

# Функция преобразования остальных категорий
def convert_linear(category, value, from_unit, to_unit):
    if category not in UNITS:
        raise ValueError("Неизвестная категория конверсии: " + str(category))

    units = UNITS[category]
    if from_unit not in units:
        raise ValueError("Неизвестная единица: " + from_unit)
    if to_unit not in units:
        raise ValueError("Неизвестная единица: " + to_unit)
    base_value = value * units[from_unit]
    result = base_value / units[to_unit]
    return result

```

## 2.2 Модульное тестирование

Описание тестов: Тестовый набор проверяет основные функции модуля конвертации: температурные шкалы, линейные преобразования, обработку ошибок и точность обратных вычислений.

*Листинг 6 — Код тестов.*

```

import pytest
import core.convert_script as conv

class TestConvertScript:

    def test_c_to_f_basic(self):
        assert conv.c_to_f(0) == 32.0
        assert conv.c_to_f(100) == 212.0

    def test_f_to_c_basic(self):
        assert conv.f_to_c(32) == 0.0
        assert conv.f_to_c(212) == 100.0

    def test_c_to_k_basic(self):
        assert conv.c_to_k(0) == 273.15
        assert conv.c_to_k(-273.15) == 0.0

    def test_k_to_c_basic(self):
        assert conv.k_to_c(273.15) == 0.0
        assert conv.k_to_c(0) == -273.15

    def test_convert_length_meter_to_km(self):
        result = conv.convert_linear("длина", 1000, "метр", "километр")

```

```

    assert result == 1.0

def test_convert_mass_gram_to_kg(self):
    result = conv.convert_linear("масса", 1000, "грамм", "килограмм")
    assert result == 1.0

def test_convert_unknown_category(self):
    with pytest.raises(ValueError):
        conv.convert_linear("объем", 100, "литр", "галлон")

def test_convert_unknown_unit(self):
    with pytest.raises(ValueError):
        conv.convert_linear("длина", 100, "фут", "метр")

def test_temperature_round_trip(self):
    original = 25.5
    converted = conv.f_to_c(conv.c_to_f(original))
    assert converted == pytest.approx(original, abs=1e-10)

def test_linear_round_trip(self):
    original = 150
    converted = conv.convert_linear("длина",
        conv.convert_linear("длина", original, "сантиметр", "метр"),
        "метр", "сантиметр")
    assert converted == pytest.approx(original, abs=1e-10)

if __name__ == "__main__":
    print(conv.c_to_f(0) == 32.0)

```

Методология: Тестирование реализовано на pytest с применением базовых проверок, обработки исключений и сравнительного анализа результатов.

Анализ покрытия кода: Тесты охватывают все ключевые функции конвертации, валидацию данных и обработку ошибок, включая позитивные и негативные сценарии.

Исправление ошибок:

Краткое описание ошибки: «Неверное преобразование Цельсия в Фаренгейты».

Статус ошибки: открыта («Open»).

Категория ошибки: серьезная («Major»).

Тестовый случай: «Проверка алгоритма функционирования программы».

Описание ошибки:

1. Загрузить программу.
  2. В поле ввода ввести строку «1».
  3. В поле ввода ввести строку «2».
  4. В поле ввода ввести строку «1».
  5. В поле ввода ввести строку «10».
4. Полученный результат: «122». Ожидаемый результат: «50».

Листинг 7 — Исправленный код.

```
# Единицы измерения
UNITS = {
    "длина": {
        "миллиметр": 10 ** -3,    # 0.001 метра
        "сантиметр": 10 ** -2,    # 0.01 метра
        "дециметр": 10 ** -1,     # 0.1 метра
        "метр": 1,                 # базовая единица
        "километр": 10 ** 3        # 1000 метров
    },
    "масса": {
        "грамм": 1,                # базовая единица
        "килограмм": 10 ** 3,      # 1000 грамм
        "центнер": 10 ** 5,       # 100 000 грамм
        "тонна": 10 ** 6           # 1 000 000 грамм
    }
}

# Функция преобразования цельсия в фаренгейты
def c_to_f(c):
    return c * 9.0 / 5.0 + 32.0

# Функция преобразования фаренгейтов в цельсии
def f_to_c(f):
    return (f - 32.0) * 5.0 / 9.0

# Функция преобразования цельсия в кельвины
def c_to_k(c):
    return c + 273.15

# Функция преобразования кельвинов в цельсии
def k_to_c(k):
    return k - 273.15

# Функция преобразования фаренгейтов в кельвины
def f_to_k(f):
    return (f - 32.0) * 5.0 / 9.0 + 273.15

# Функция преобразования кельвинов в фаренгейты
def k_to_f(k):
    return (k - 273.15) * 9.0 / 5.0 + 32.0

# Функция преобразования остальных категорий
def convert_linear(category, value, from_unit, to_unit):
    if category not in UNITS:
```

```

raise ValueError("Неизвестная категория конверсии: " + str(category))

units = UNITS[category]
if from_unit not in units:
    raise ValueError("Неизвестная единица: " + from_unit)
if to_unit not in units:
    raise ValueError("Неизвестная единица: " + to_unit)
base_value = value * units[from_unit]
result = base_value / units[to_unit]
return result

```

## 2.3 Мутационное тестирование

Создание мутантов:

1. Мутации в функциях конвертации температур:

| Функция   | Исходный код             | Мутант (Ошибка)                                   | Результат теста                                    | Убит ? | Недостаток теста? |
|-----------|--------------------------|---------------------------------------------------|----------------------------------------------------|--------|-------------------|
| c_to_f(c) | $c * 9.0 / 5.0 + 32.0$   | $c * 9.0 / 5.0 + 32.1$<br>(Изменение константы)   | test_c_to_f_basic<br>Провалится (32.1 != 32.0)     | Да     | Нет               |
| f_to_c(f) | $(f - 32.0) * 5.0 / 9.0$ | $(f - 32.0) * 5.0 / 9.1$<br>(Изменение константы) | test_f_to_c_basic<br>Провалится                    | Да     | Нет               |
| c_to_k(c) | $c + 273.15$             | $c + 273.16$<br>(Изменение константы)             | test_c_to_k_basic<br>Провалится (273.16 != 273.15) | Да     | Нет               |
| k_to_c(k) | $k - 273.15$             | $k + 273.15$<br>(Замена оператора)                | test_k_to_c_basic<br>Провалится                    | Да     | Нет               |
| k_to_f(k) | $(k - 273.15)$           | $(k - 273.15) * 9.0 / 5.0 + 32.0$                 | Выживет при                                        | Нет    | Да                |



| Функция   | Исходный код                      | Мутант (Ошибка)                                      | Результат теста     | Убит ? | Недостаток теста? |
|-----------|-----------------------------------|------------------------------------------------------|---------------------|--------|-------------------|
|           | $* 9.0 / 5.0 + 32.0$              | $9.0 / 5.0 - 32.0$<br>(Замена + на -)                | k=273.15!           |        |                   |
| f_to_k(f) | $(f - 32.0) * 5.0 / 9.0 + 273.15$ | $(f - 32.0) * 5.0 / 9.0 - 273.15$<br>(Замена + на -) | Выживет при f=32.0! | Нет    | Да                |

## 2. Мутации в Функции convert\_linear:

| Код | Исходный код                          | Мутант (Ошибка)                                | Результат теста                                                                   | Убит ? | Недостаток теста? |
|-----|---------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------|--------|-------------------|
| A   | if category not in UNITS:             | if category in UNITS:<br>(Замена not in на in) | test_convert_unknown_category<br>Провалится (ожидали ValueError, но его не будет) | Да     | Нет               |
| B   | raise ValueError(...)                 | Удалить строку (Удаление оператора)            | test_convert_unknown_category<br>Провалится (тест ожидает исключение, но его нет) | Да     | Нет               |
| C   | base_value = value * units[from_unit] | base_value = value + units[from_unit]          | test_convert_length_meter_to_km<br>Провалится (1000*1 != 1000+1)                  | Да     | Нет               |

| К<br>о<br>д | Исходный код                               | Мутант<br>(Ошибка)                                                  | Результат<br>теста                                                                                | Убит<br>? | Недоста<br>ток<br>теста? |
|-------------|--------------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------|--------------------------|
|             |                                            | (Замена *<br>на +)                                                  |                                                                                                   |           |                          |
| D           | result =<br>base_value /<br>units[to_unit] | result<br>=<br>base_value *<br>units[to_unit]<br>(Замена /<br>на *) | test_convert_length_<br>meter_to_k<br>m Провалится<br>(1000/1000 !=<br>1000*1000)                 | Да        | Нет                      |
| E           | if from_unit<br>not in units:              | if<br>from_unit<br>in<br>units:<br>(Замена<br>not in<br>на in)      | test_convert_<br>unknown_<br>unit<br>Провалится<br>(ожидали<br>ValueError,<br>но его не<br>будет) | Да        | Нет                      |

Анализ их выживания:

Тесты test\_c\_to\_f\_basic и test\_f\_to\_c\_basic хорошо покрывают крайние точки (0°C/32°F и 100°C/212°F). Однако, для функций k\_to\_f и f\_to\_k, мутанты, меняющие финальный знак (+ 32.0 на - 32.0 и + 273.15 на - 273.15), выжили при входном значении, которое обнуляет первую часть формулы (32°F или 273.15K).

Корректировка тестов:

1. Тест для k\_to\_f: Проверить известную точку, где первая часть формулы не равна нулю.

*Листинг 16 — Код теста.*

```
def test_k_to_f_basic(self):
    # 0K = -459.67F
    assert conv.k_to_f(0) == pytest.approx(-459.67, abs=1e-10)
    # 288.15K = 59.0F
    assert conv.k_to_f(288.15) == 59.0
```

2. Тест для `f_to_k`: Проверить известную точку, где разность не равна нулю.

*Листинг 17 — Код теста.*

```
def test_f_to_k_basic(self):  
    # 32F = 273.15K  
    assert conv.f_to_k(32) == 273.15  
    # 59F = 288.15K  
    assert conv.f_to_k(59) == 288.15
```

## 3 АНАЛИЗ И ВЫВОДЫ

### 3.1 Оценка качества тестирования

Итоговое Качество Тестирования:

1. Полнота (Покрытие): Теперь тесты обеспечивают не только высокое покрытие строк кода, но и высокое покрытие логики. Почти все значимые операторы (математические, логические, сравнения) проверяются на их чувствительность к ошибкам.
2. Надежность (Чувствительность): Набор тестов способен обнаруживать даже атомарные, синтаксически небольшие ошибки (мутанты), которые могли бы быть допущены программистом, что является прямым доказательством его надежности.
3. Тестирование интеграционных точек: Особое внимание уделено интеграционным точкам (например, view вызывает validator и history). Проверка, что удаление валидации приводит к сбою, а удаление записи в историю обнаруживается, делает интеграционные тесты более надежными.

### 3.2 Анализ недостатков и их устранение

Улучшения, которые были достигнуты благодаря мутационному тестированию:

| Модуль         | Ранее Выжившие Мутанты (Пробелы)                                          | Статус после изменений                                                                     |
|----------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| convert_script | Формулы k_to_f и f_to_k не проверялись для всех ненулевых входных данных. | Убиты. Добавлены явные тесты для k_to_f и f_to_k, проверяющие ненулевые контрольные точки. |

### **3.3 Итоговые выводы по результатам работы**

В результате выполнения практической работы были достигнуты поставленные цели: освоены методы модульного и мутационного тестирования, разработаны и проанализированы тестовые наборы. В ходе исследования успешно применено мутационное тестирование, созданы эффективные модульные тесты с высоким покрытием кода. Мутационное тестирование позволило выявить и устранить пробелы в тестовом наборе, повысив качество проверки программного продукта. Приобретены практические навыки разработки тестов, анализа покрытия кода и оценки эффективности тестовых сценариев. Полученные результаты подтверждают, что комбинированный подход к тестированию обеспечивает надёжную проверку качества программного обеспечения и помогает обнаруживать потенциальные ошибки на ранних этапах разработки.